SCALE FOR PROJECT CALL ME MAYBE (/PROJECTS/CALL-ME-MAYBE)

You should evaluate 1 student in this team

★

Git repository

```
git@vogsphere.42abudhabi.ae:vogsphere/intra-uuid-b4cb5bcd-7624-406b
```

Introduction

- Remain polite, courteous, respectful and constructive throughout the evaluation process. The well-being of the community depends on it.

- Identify with the person (or the group) evaluated the eventual dysfunctions of the work. Take the time to discuss and debate the problems you have identified.

- You must consider that there might be some difference in how your peers might have understood the project's instructions and the scope of its functionalities. Always keep an open mind and grade him/her as honestly as possible. The pedagogy is valid only and only if peer-evaluation is conducted seriously.

Guidelines

- Only grade the work that is in the student or group's GiT repository.

- Double-check that the GiT repository belongs to the student or the group. Ensure that the work is for the relevant project and also check that "git clone" is used in an empty folder.

- Check carefully that no malicious aliases was used to fool you and make you evaluate something other than the content of the official repository.

- To avoid any surprises, carefully check that both the evaluating and the evaluated students have reviewed the possible scripts used to facilitate the grading.

- If the evaluating student has not completed that particular project yet, it is mandatory for this student to read the entire subject prior to starting the defence.

- Use the flags available on this scale to signal an empty repository, non-functioning program, a norm error, cheating etc. In these cases, the grading is over and the final grade is 0 (or -42 in case of cheating). However, with the exception of cheating, you are encouraged to continue to discuss your work (even if you have not finished it) in order to identify any issues that may have caused this failure and avoid repeating the same mistake in the future.

- Remember that for the duration of the defence, no segfault, no other unexpected, premature, uncontrolled or unexpected termination of the program, else the final grade is 0. Use the appropriate flag.
You should never have to edit any file except the configuration file if it exists. If you want to edit a file, take the time to explicit the reasons with the evaluated student and make sure both of you are okay with this.

- You must also verify the absence of memory leaks. Any memory allocated on the heap must

be properly freed before the end of execution.
You are allowed to use any of the different tools available on the computer, such as leaks, valgrind, or e_fence. In case of memory leaks, tick the appropriate flag.

Attachments

moulinette.zip (https://cdn.intra.42.fr/document/document/57710/moulinette.zip)

subject.pdf (https://cdn.intra.42.fr/pdf/pdf/229078/en.subject.pdf)

data.zip (https://cdn.intra.42.fr/document/document/57711/data.zip)

llm_sdk.zip (https://cdn.intra.42.fr/document/document/57712/llm_sdk.zip)

Mandatory Part

Preliminaries

Check the following requirements:

- Only grade the work that is in the student's or group's Git repository.
- The project must be run using `uv run python -m src`
- All errors should be handled gracefully without crashing
- Verify that the output JSON follows the exact format specified
- Check that constrained decoding is implemented (not just prompting)
- Ensure near-perfect JSON validity (100% parseable output)

Yes No

Project Structure and Dependencies

Verify the project setup:

- Run `uv sync` successfully
- Verify that llm_sdk is properly integrated
- Check that all classes use pydantic for validation
- Ensure the program can be run with `uv run python -m src`
- Verify input/ directory structure is correct
- Check that output/ directory is created during execution

Yes No

Input File Handling

Test input file processing:

- Verify the program correctly reads function_calling_tests.json
- Verify the program correctly reads functions_definition.json
- Test with invalid JSON in input files (should handle gracefully)
- Test with missing input files (should provide clear error messages)
- Verify proper error handling without crashes

Yes No

Output File Format

Verify output file correctness:

- Check that the output file is created (default: data/output/function_calls.json, or the path provided with --outpu
- Verify the file contains 100% valid and retrievable JSON (no syntax or parsing errors)
- Confirm that the JSON strictly follows the expected schema
- Check that each entry includes exactly: prompt, name, and parameters keys
- Ensure all required arguments are present and match the defined schema
- Verify that argument types and allowed values comply with the function specifications (e.g., firmware field re predefined options)
- For parameters whose type is "number", the value must be a JSON floating-point value (e.g. 2.0, not 2)
- Confirm there are no extra keys, text, or prose outside the JSON structure

Yes No

Function Calling Accuracy

Evaluate function calling accuracy:

- Test with simple prompts (e.g., "add 2 and 3")
- Verify correct function selection (>90% accuracy expected)
- Check argument extraction accuracy (>90% expected)
- Test with ambiguous prompts
- Verify the system handles edge cases (empty strings, large numbers)

Does the solution achieve at least 90% accuracy on function selection?

Yes No

LLM SDK Usage

Verify proper LLM SDK usage:

- Check that SDK methods are used according to the SDK implementation
- Ensure no private methods or attributes are accessed
- Confirm the Qwen/Qwen3-0.6B model is used

Yes No

Error Handling and Robustness

Test error handling:

- Test with malformed input JSON
- Test with missing function definitions
- Test with prompts that don't match any function
- Verify clear error messages are provided
- Ensure the program never crashes unexpectedly

Does the program handle all error cases gracefully?

Yes No

Performance and Reliability

Evaluate performance:

- Check that all test prompts are processed in reasonable time (<5 minutes)
- Verify 100% of outputs are valid JSON (parseable)
- Check that the solution achieves >90% accuracy on provided tests
- Verify the system is reliable across multiple runs

Does the solution meet these performance criteria?

Yes No

Code Quality and Documentation

Review code quality:

- Check that code is well-organized and readable
- Verify proper use of pydantic for validation
- Check that README.md explains the algorithm clearly
- Verify README includes design decisions and challenges
- Check for proper type hints and documentation

Is the code quality and documentation satisfactory?

Yes No

Moulinette Evaluation

Run the moulinette evaluation:

Follow this instructions carefully:

1. Navigate to the moulinette directory
2. Run `uv sync` successfully
3. Run `uv run python -m moulinette prepare_exercises --set private` successfully

4. Run the student's program on the generated inputs to produce the answers file, using the files generated in step
   `functions_definition` and `--input`, e.g.: `uv run python -m src --functions_definition <moulinette_dir>/data/input/functions_definition.json --input <moulinette_dir>/data/input/function_calling_tests.json --output <moulinette_dir>/data/input/function_calls.json`
5. Run `uv run python -m moulinette grade_student_answers --set private --student_ans` `data/input/function_calls.json` successfully
6. Verify the total score is greater than 0
7. Check that the evaluation completes without errors

Does the moulinette evaluation pass successfully?

☑ Yes ☒ No

Bonus

Check for bonus features (optional, not required for passing):

- Support for multiple LLM models beyond Qwen/Qwen3-0.6B
- Recoding the tokenizer: Not using encode and decode methods in the main code, but using get_logits_from_in and get_path_to_vocab_file.
- Advanced error recovery mechanisms
- Performance optimizations (caching, batching), PyTorch may be used exclusively for tensor manipulation and when interacting with the provided llm_sdk.
- Comprehensive test suite
- Visualization of the generation process
- Support for complex nested function arguments
- Public implementation of tokenizer encode and optional decode methods compatible with the vocabulary file p by the SDK.
- Demonstration of how encoding and decoding integrate with constrained decoding

Are there any notable bonus features implemented?

Rate it from 0 (failed) through 5 (excellent)

Ratings

Don’t forget to check the flag corresponding to the defense

✔ Ok ★ Outstanding project

Empty work Incomplete work Invalid compilation Norme Cheat

▲ Concerning situation ⊘ Forbidden function Can’t support / explain code

Conclusion

Leave a comment on this evaluation (2048 chars max)

Finish evaluation

API General Terms of Use (https://profile.intra.42.fr/legal/terms/33) Declaration on the use of cookies (https://profile.intra.42.fr/legal/terms/2) Privacy policy (https://profile.intra.42.fr/legal/terms/5) General term of use of the site (https://profile.intra.42.fr/legal/terms/6) Rules of procedure (https://profile.intra.42.fr/legal/ter